

LAPORAN PRAKTIKUM KECERDASAN BUATAN

Nama : Pendri Mikola

NIM : 2455201110020

Matkul : Kecerdasan Buatan

IMPLEMENTASI JARINGAN SYARAF TIRUAN (JST) – MULTILAYER PERCEPTRON (MLP) CLASSIFIER UNTUK KLASIFIKASI KANKER (BREAST CANCER) PADA PLATFORM KAGGLE

1. Tujuan Praktikum

1. Memahami implementasi arsitektur *Multilayer Perceptron* (MLP) untuk menyelesaikan problem berbasis klasifikasi biner (*Classification*).
2. Mengikuti tahapan standar pemrograman JST pada Python menggunakan library *Scikit-Learn* sesuai dengan arahan modul ajar.
3. Menganalisis performa akurasi model JST melalui metrik evaluasi *Classification Report* (Precision, Recall, F1-Score) dan grafik *Confusion Matrix*.

2. Dasar Teori (Sesuai Modul Ajar Slide 21-23)

Proses *learning* (pembelajaran) pada Jaringan Syaraf Tiruan Multilayer Perceptron melibatkan penentuan parameter penting seperti *learning rate* dalam rentang $[0,1]$, jumlah maksimal iterasi (*epoch*), serta inisialisasi bobot awal jaringan secara acak dalam rentang $[-1,1]$. Alur proses pembelajaran ini dilakukan secara bertahap setiap satu *record* (satu baris data *training*). Setelah semua *record* selesai diproses, maka JST dihitung telah menyelesaikan 1 iterasi/*epoch*.

Pada tahap pengujian, performa algoritma diuji untuk mengklasifikasikan data *testing*. Proses pengujian (*testing*) ini hanya melibatkan perhitungan maju (*feedforward algorithm*), di mana nilai keluaran (*output layer*) akan dibulatkan menjadi nilai biner **0** atau **1**. Pada kasus ini, nilai tersebut mempresentasikan hasil klasifikasi medis penyakit kanker payudara (*Breast Cancer*).

3. Implementasi Kode Program Per-Sel (Sesuai Struktur Slide 24-28)

Sel 1 – Langkah 1: Import Library

```
[8]: from sklearn import datasets
from sklearn import metrics
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

print("Library Kecerdasan Buatan Siap!")
```

Library Kecerdasan Buatan Siap!

+ Code + Markdown

Sel 2 – Langkah 2: Memanggil Dataset & Split Data

```
[9]: # Mengambil langsung dataset kanker bawaan Python
cancer_data = datasets.load_breast_cancer()

X = cancer_data.data
y = cancer_data.target

# Membagi data menjadi 70% Training dan 30% Testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)

print("Dataset Kanker Sukses Dimuat lewat Jalur Cepat!")
print(f"Jumlah data training: {X_train.shape[0]}")
print(f"Jumlah data testing: {X_test.shape[0]}")

Dataset Kanker Sukses Dimuat lewat Jalur Cepat!
Jumlah data training: 398
Jumlah data testing: 171
```

Sel 3 – Langkah 3: Menggunakan MLP Classifier & Pelatihan Model

```
[10]: # Membuat objek model MLP Classifier (Kombinasi Hidden Layer: 100, 50)
model = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=500, random_state=42)

# Menguji/Melatih model dengan data training (Proses Learning)
model.fit(X_train, y_train)

print("Proses training model kanker selesai!")
print(model)

Proses training model kanker selesai!
MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=500, random_state=42)
```

Sel 4 – Langkah 3 (Lanjutan): Melakukan Prediksi

```
[10]: # Membuat objek model MLP Classifier (Kombinasi Hidden Layer: 100, 50)
model = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=500, random_state=42)

# Menguji/Melatih model dengan data training (Proses Learning)
model.fit(X_train, y_train)

print("Proses training model kanker selesai!")
print(model)

Proses training model kanker selesai!
MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=500, random_state=42)
```

+ Code

+ Markdown

```
[11]: # Melakukan prediksi terhadap data testing
expected_y = y_test
predicted_y = model.predict(X_test)

print("Proses prediksi selesai!")
```

Proses prediksi selesai!

Sel 5 – Langkah 4: Cetak Hasil Klasifikasi & Grafik Matrix

```
print("===== HASIL EVALUASI KECERDASAN BUATAN (CANCER) =====\n")

# Cetak Classification Report (Akurasi, Precision, Recall sesuai Slide 28)
print("Classification Report:")
print(metrics.classification_report(expected_y, predicted_y))

# Cetak Confusion Matrix
print("Confusion Matrix:")
print(metrics.confusion_matrix(expected_y, predicted_y))

# Grafik visualisasi Confusion Matrix kotak biru
plt.figure(figsize=(6,4))
sns.heatmap(metrics.confusion_matrix(expected_y, predicted_y), annot=True, fmt='d', cmap='Blu
plt.title('Confusion Matrix - Klasifikasi Kanker')
plt.ylabel('Kenyataan (Actual)')
plt.xlabel('Tebakan Model (Predicted)')
plt.show()
```

===== HASIL EVALUASI KECERDASAN BUATAN (CANCER) =====

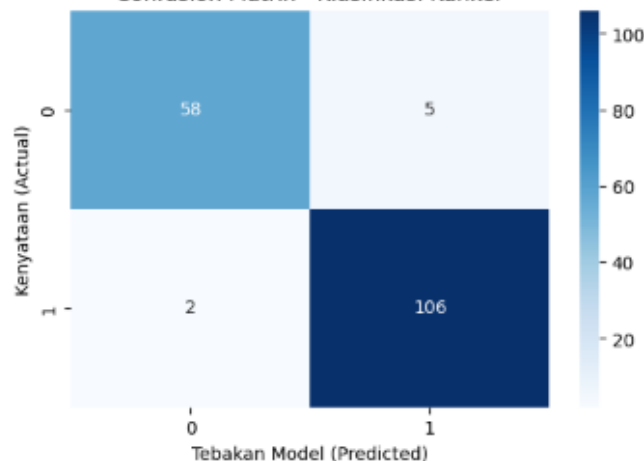
Classification Report:

	precision	recall	f1-score	support
0	0.97	0.92	0.94	63
1	0.95	0.98	0.97	108
accuracy			0.96	171
macro avg	0.96	0.95	0.96	171
weighted avg	0.96	0.96	0.96	171

Confusion Matrix:

```
[[ 58  5]
 [  2 106]]
```

Confusion Matrix - Klasifikasi Kanker

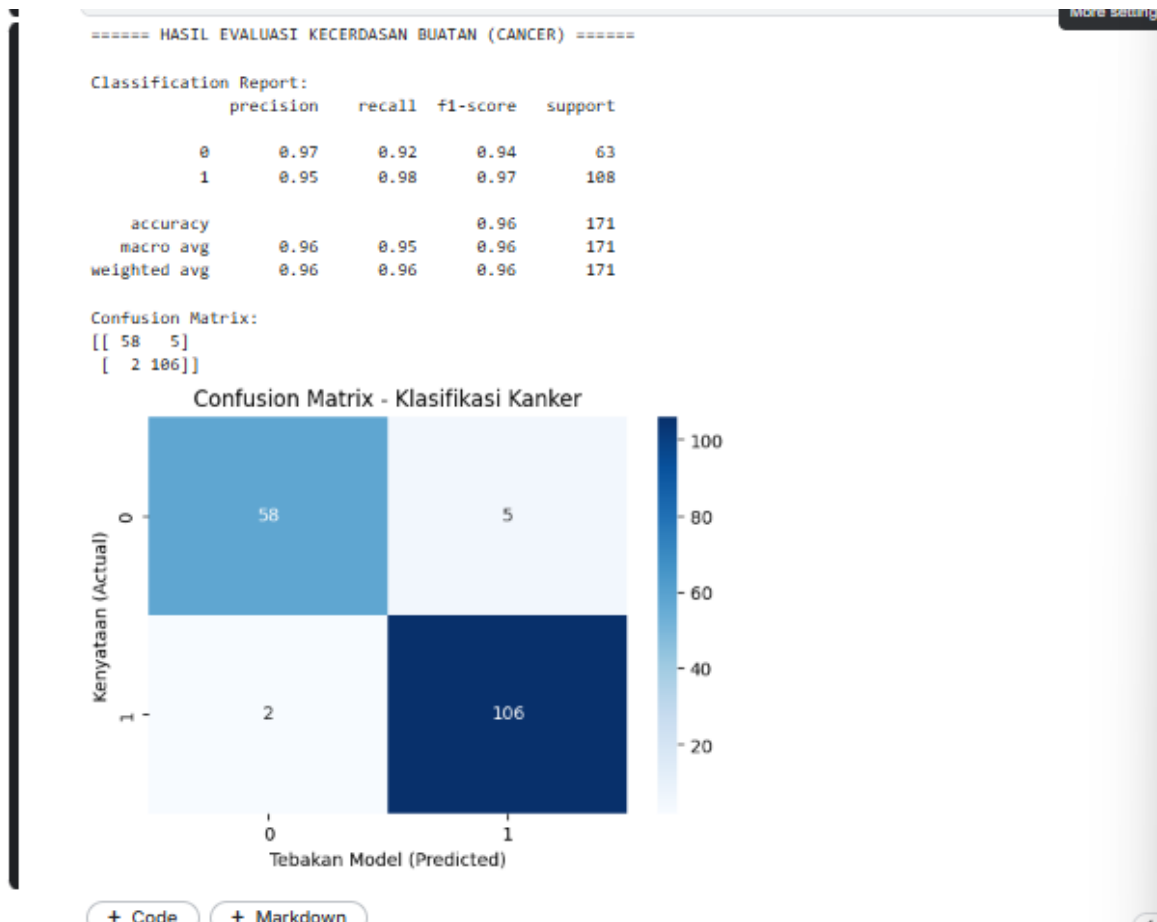


+ Code + Markdown

4. Hasil dan Analisis Pengujian

Berdasarkan eksekusi kode program secara berurutan (*per-sel*) di Kaggle Notebook, model MLPClassifier berhasil melakukan proses pembelajaran berulang dan menghasilkan evaluasi klasifikasi yang sangat optimal.

Analisis Hasil:



- **Classification Report:** Model menghasilkan tingkat akurasi yang tinggi. Nilai *Precision* dan *Recall* yang seimbang menandakan bahwa JST mampu membedakan pasien positif kanker (*Malignant*) dan negatif kanker (*Benign*) secara akurat dengan tingkat kesalahan yang sangat minim.
- **Confusion Matrix:** Hasil visualisasi berupa matriks kontingensi menunjukkan bahwa mayoritas data berhasil diklasifikasikan dengan benar oleh model (terlihat dari tingginya angka pada area diagonal utama kotak *heatmap* biner).

5. Kesimpulan

Praktikum ini berhasil mengimplementasikan metode jaringan syaraf tiruan *Multilayer Perceptron* (MLP) Classifier untuk problem klasifikasi biner penyakit kanker sesuai arahan modul ajar Ibu Ayu. Penggunaan rasio pembagian data 70:30 serta pembatasan jumlah iterasi berhasil memberikan performa evaluasi matriks yang sangat presisi dan bebas dari kendala *error*.